

Project 1 on Machine Learning: regression analysis, resampling methods and gradient descent

Tim Herrmann Holmen

FYS-STK4155

Department of Physics

This project explores the data analysis methods that have contributed to the development of modern machine learning. The main objective is to study regression methods in detail, including ordinary least squares, Ridge regression and Lasso regression. Resampling techniques, such as bootstrap and cross-validation, allow us to evaluate and select the most suitable method for the problem. To optimise the model parameters, we use gradient descent and investigate different methods for performing derivative-based optimisation. Cross-validation helps select hyperparameters, such as polynomial degree and regularisation strength. We have studied Newton's method and automatic differentiation to support the optimisation of model parameters. The polynomial models are fitted to a specific one-dimensional function, known as the Runge function, with added noise, providing a known analytical function against which the predictions can be compared.

CONTENTS

I. Introduction	1	B. Ridge	19
A. 2a The motivation.	1	1. Regularization	19
B. 2b What has been done and what i do.	2	2. Measure of Quality	19
C. The structure of the report	2	C. Bias-Variance tradeoff and Bootstrap	20
II. Methods	3	D. K-fold Cross validation	20
A. The data	3	E. Gradient Descent	20
1. Data processing: Splitting and scaling	3	1. Plain Gradient descent	20
B. Linear Regression	3	2. Advanced gradient descent	20
1. Ordinary Least Squares	3	F. Lasso gradient descent	20
2. Pseudocode OLS	4	G. Final analysis	20
C. Ridge Regression	4	V. Conclusion	21
1. Pseudocode Ridge	4	VI. LLM Declaration	21
D. Lasso Regression	4	References	21
E. Evaluation	5		
1. Measures of quality	5		
2. Bias-Variance tradeoff	5		
3. Bootstrap	5		
4. Kfold Cross Validation	6		
F. Gradient descent	6		
1. Plain gradient descent	6		
2. Pseudocode GD	7		
3. Lasso Gradient descent	8		
G. Experimental setup	8		
III. Results	9		
A. The data	9		
B. OLS and Ridge regression	9		
1. OLS	9		
2. Ridge	10		
C. Bias-Variance tradeoff and bootstrap	12		
1. Fig 2.11	12		
2. Bootstrap	13		
D. K-fold Cross validation	13		
E. Gradient descent and optimisation methods	15		
1. Automatic differentiation	15		
2. Plain Gradient descent	15		
3. Advanced gradient descent	16		
F. Lasso with gradient descent	17		
G. Final cross-validation comparison	18		
IV. Discussion	19		
A. OLS	19		
1. Parameter plot	19		
2. Measure of quality	19		

I. INTRODUCTION

A. 2a The motivation.

The reason we want to fit a function to data is that we want to predict the value of the function to new unknown data points. For fitting a function to data points, we use polynomials, and we want to find the best polynomial that fits the data. The best polynomial is the one that minimizes the mean squared error (MSE) between the predicted values and the true values (for the known data). The MSE can be decomposed into three components: bias, variance and irreducible error.

$$\text{MSE} = \text{Bias}^2 + \text{Var}[\hat{y}] + \text{irreducible noise}$$

The bias measures how far the average prediction is from the truth. The variance measures how much the prediction moves when the training set is redrawn. The irreducible error is the variance of the noise itself, which is baked into the data such that the model cannot predict it. (Hjorth-Jensen, 2016, p. 62) The bias-variance trade-off is the balance between bias and variance, in which we minimize the MSE. We do not want an overfitted model,

which has low bias and high variance, nor do we want an underfitted model, which has high bias and low variance. Optimally, we want a model that is just right, with low bias and low variance. When fitting a polynomial to data, we want to find the right degree of the polynomial and the right parameters for each degree. For example if there is a linear relationship between the data points, and we can assume this by looking at the data, we use a second order polynomial to describe the data, and the parameters are the coefficients of the polynomial, which can be changed in accordance with the relationship between the data points. The parameters are found by minimizing the MSE between the model prediction and datapoints, for avoiding overfitting, we can split the data into training and testing sets, to see how well the model performs on unseen data. We can use regularization methods, such as ridge regression and lasso regression, to regularize the model and avoid overfitting, and thus lower the variance, while maintaining a good bias. We can also use resampling methods, such as bootstrap and k-fold cross validation, to estimate the performance of the model on unseen data, and thus choose the best model for the data.

With lasso regression, we do not have the analytical solution for the gradient, as we do for OLS and Ridge, and we have to use gradient descent to find the optimal parameters. Gradient descent is an optimization algorithm inspired by newtons method, which is used to find the minimum of a function. The function we want to minimize is the cost function, which is the MSE between the predicted values and the true values. The gradient descent algorithm uses the derivative of the cost function to find the direction of steepest descent, and then take steps in that direction, with a step size determined by the learning rate. We can use different optimization algorithms, such as momentum, adagrad, rmsprop and adam, where the learning rate of the descent is dependent on the iteration number to improve the convergence of gradient descent. After exploring the different methods, we can compare them with cross validation, and choose the best model for the data.

B. 2b What has been done and what i do.

The linear models with OLS and Ridge have been implemented, and both have closed form solutions, thus they can be solved without gradient descent. Lasso regression has been implemented, but it does not have a closed form solution, thus we have to use gradient descent to find the optimal parameters. The project starts with the implementation of OLS and Ridge regression, OLS uses the least square distance between model and data to find the optimal parameters, this is the cost function, and the optimal parameters are found by minimizing the cost function. Ridge regression does the same, but adds a regularization term to the cost function, which penalizes large

parameters, and thus avoids overfitting. What ridge removes in variance, it adds in increased bias, which will be discussed as the bias-variance tradeoff. For exploring this theme we have employed resampling methods, the bootstrap and K-fold cross validation, to estimate the performance of the model on unseen data, and thus choose the best model for the data. The linear models OLS and Ridge are first treated separately, and then compared to each other with resampling methods to find the best degree of the polynomial, and the best regularization parameter, λ , for Ridge. For lasso regression, we have to use gradient descent to find the optimal parameters, since it does not have a closed form solution. With this end in mind, we have implemented plain gradient descent, inspired by Newton's iteration method for solving differentials, and explored different optimization algorithms, such as momentum, adagrad, rmsprop and adam, where the learning rate of the descent is dependent on the iteration number. These methods are first implemented on OLS and Ridge, and compared to the closed form solutions of both, to see how well they perform, in terms of convergence (towards zero difference between closed form and gradient descent solutions), and data cost (number of iterations). In terms of hyperparameters, Ridge and Lasso both have a regularization term. The degree is a hyperparameter for all three models, and the learning rate is a hyperparameter for gradient descent. This project can be split in two parts, where first the Ridge and ordinary least square regression methods are used and evaluated. Then gradient descent is developed and used on all three methods, including Lasso, and the three methods are evaluated by k-fold cross validation. Underway the analytical solutions and `sci-kit learn` library are used and compared, to ensure numerical precision.

C. The structure of the report

The Introduction presents the motivation and objectives of the project. The Methods section describes the data preparation, OLS, Ridge and Lasso regression, evaluation metrics, resampling techniques, optimisation algorithms and experimental setup. The Results section presents the regression performance, bias-variance analysis, bootstrap and cross-validation results, gradient verification and convergence experiments, followed by the final comparison of OLS, Ridge and Lasso. The Discussion section interprets the results and examines the strengths and limitations of the methods. The Conclusion summarises the main findings concerning model complexity, regularisation and optimisation. The LLM Declaration describes how language models assisted with the code and report.

The project code is available in the GitHub repository ([Holmen, 2026](#)).

II. METHODS

A. The data

In this project, the input data is normally distributed, and the output data is generated by Runge's function, with added noise, Runge Function which is a one-dimensional function of second order.

$$f(x) = \frac{1}{1 + 25x^2}$$

From the beginning, we have created a relation between the input and output data. This is helpful since the project is research in machine learning methods, it is usefull to know the exact solution. The noise is normally distributed, with a mean of 0 and a variable standard deviation. From the form of the runge function, we can see that the function has a maximum at $x=0$, and the function is symmetric around this point, for values larger than $[-1,1]$, the function approaches zero. Thus we will focus on data which is normally distributed between -1 and 1. The output is generated by adding noise σ to the runge function defined above, and the noise is normally distributed with a mean of 0 and a standard deviation of σ .

The input value x is transformed into a design matrix X , where the columns describe a polynomial of variable degree d , in such a way that the first column is x^0 , the intercept, the second column is x^1 , and so on, up to x^d . The rows of the matrix are the data points, and are thus copied and multiplied according to the polynomial degree, or the column of the matrice.

$$\mathbf{X} = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^d \\ 1 & x_2 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^d \end{pmatrix} \in \mathbb{R}^{n \times (d+1)}. \quad (1)$$

1. Data processing: Splitting and scaling

- Train test split of skicit
- Unless explicitly told otherwise, such as in the bootstrap resampling method, the training and test input data are normalized, using training values.
- All random data are within the random seed 2026

a. Pseudocode Data Here is a pseudocode for the splitting and scaling of the data:

Listing 1 Preparation of training and test data.

```
FUNCTION PrepareData(x, y, degree):
    X <- DesignMatrix(x, degree)
```

```
Split X and y into:
    70% training data
    30% test data
Use random seed 2026

mu <- column means of X_train
s <- column standard deviations of X_train

mu[0] <- 0
s[0] <- 1

X_train <- (X_train - mu) / s
X_test <- (X_test - mu) / s

RETURN X_train, X_test,
        y_train, y_test
```

B. Linear Regression

The basic idea of linear regression is, by using a polynomial $f(x)$ of fixed degree to describe the data points given the input data x , to adjust the coefficients in such a way that the difference between the predicted values of the model $f(x)$ and the true output value $y(x)$. The coefficients are adjusted so that mean of the sum of the difference squared is minimised, which is the basis for the name least squares regression.

1. Ordinary Least Squares

Thus the cost function, for least squared, is written mathematically:

$$C(\theta) = \frac{1}{n} \|y - X\theta\|_2^2 \quad (2)$$

(Hjorth-Jensen, 2016, p. 81) This is the optimization problem, to adjust the parameter θ such that the cost function is minimized. The cost function is convex, and the minima is in the extremum point the $\nabla_{\theta} C(\theta) = 0$. The gradient is

$$\nabla_{\theta} C_{OLS}(\theta) = \frac{2}{n} X^T (X\theta - y). \quad (3)$$

Setting this gradient to zero gives the normal equations:

$$\hat{\theta} = (X^T X)^{-1} X^T y \quad (4)$$

(Hjorth-Jensen, 2016, p. 81) Through singular value decomposition (Hjorth-Jensen, 2016, p. 42), it can be shown that the best estimate for this theta, which minimize the cost function, is the pseudoinverse of X multiplied by the output data y ,

$$\hat{\theta} = X^+ y \quad (5)$$

(Hjorth-Jensen, 2016, p. 82) The pseudoinverse is solved by the numpy package `np.linalg.pinv(X)`, on the assumption that the columns of X are linearly independent, such that $X^T X = I$ and $X^+ = (X^T X)^{-1} X^T = I^{-1} X^T = X^T$.

It is safe to assume that the columns are linearly independent, since the number of columns (degrees d) is much smaller than the number of rows (data points n), the possibility of one column being a linear combination of the other columns is close to zero.

The data is split into training and test data as described above, and the training data is normalised as explained above. This ensures that the model can predict on the test data of X, and the predictions can be used against the test data of y to calculate the mean squared error MSE and coefficient of determination R^2 , how these values are calculated will be explained in [measures of quality](#). This procedure is done for different degrees of the polynomial design matrix. The parameters are plotted against degree, and the predicted models for different degrees are plotted against the runge function, for different values of the noise σ .

2. Pseudocode OLS

Here we go

Listing 2 Ordinary least squares regression.

```
FOR degree = 0, ..., 15:
  X_train, X_test, y_train, y_test
    <- PrepareData(x, y, degree)

  theta <- pseudoinverse(X_train) * y_train

  prediction <- X_test * theta

  mse <- MSE(y_test, prediction)
  r2 <- R2(y_test, prediction)

  Store degree, theta, mse and r2
```

C. Ridge Regression

Ridge regression adds a penalty on the squared length of the parameter vector, and the cost function is;

$$C_{\text{Ridge}}(\theta) = \frac{1}{n} \|y - X\theta\|_2^2 + \lambda \|\theta\|_2^2, \quad \lambda \geq 0. \quad (6)$$

with $\|\theta\|_2^2 = \sum_j \theta_j^2$

The gradient for this cost function is given by

$$\nabla_{\theta} C_{\text{Ridge}}(\theta) = \frac{2}{n} X^T (X\theta - y) + 2\lambda\theta. \quad (7)$$

Setting this to zero and solving for theta, ignoring the factor 1/n, gives the closed form solution to this problem

$$\hat{\theta}_{\text{Ridge}} = (X^T X + \lambda I)^{-1} X^T y \quad (8)$$

I is the $p \times p$ identity matrix. The penalty adds λ to the eigenvalues of $X^T X$. ([Hjorth-Jensen, 2016](#), p. 87)

Supposing the design matrix is orthonormal, $X^T X = I$: then $\hat{\theta}_{\text{OLS}} = X^T y$ and

$$\hat{\theta}_{\text{Ridge}} = (I + \lambda I)^{-1} X^T y = \frac{1}{1 + \lambda} \hat{\theta}_{\text{OLS}} \quad (9)$$

So every coefficient is scaled by the same factor, and the estimator tends to zero as $\lambda \rightarrow \infty$. Ridge shrinks all coefficients proportionally; it never sets any of the exactly to zero.

1. Pseudocode Ridge

Here we go:

Listing 3 Ridge regression without penalizing the intercept.

```
lambdas <- logspace(-3, 2, 5)

FOR degree = 0, ..., 15:
  X_train, X_test, y_train, y_test
    <- PrepareData(x, y, degree)

  P <- IdentityMatrix(degree + 1)
  P[0, 0] <- 0

  FOR lambda in lambdas:
    A <- transpose(X_train) * X_train
      + lambda * P

    b <- transpose(X_train) * y_train

    theta <- pseudoinverse(A) * b

    prediction <- X_test * theta

    mse <- MSE(y_test, prediction)
    r2 <- R2(y_test, prediction)

    Store degree, lambda, theta, mse and r2
```

D. Lasso Regression

Replacing the 2-norm penalty of Ridge by a 1-norm gives the cost function

$$C_{\text{Lasso}}(\theta) = \frac{1}{n} \|y - X\theta\|_2^2 + \lambda \|\theta\|_1, \quad (10)$$

$$\|\theta\|_1 = \sum_{j=0}^{p-1} |\theta_j|$$

Which is the Lasso - least absolute shrinkage and selection operator. The Lasso sets coefficients exactly to zero, and thereby performs variable selection as part of the fitting. ([Hjorth-Jensen, 2016](#), p. 90) The lasso has no closed form solution, since differentiating the [cost function](#), requires the derivative of the absolute value of θ :

$$\frac{d|\theta|}{d\theta} = \text{sgn}(\theta) = \begin{cases} +1, & \theta > 0, \\ -1, & \theta < 0. \end{cases} \quad (11)$$

This equation is undefined at the origin, $\theta = 0$, thus the closed form solution is lost, but a solution can still be found through the soft thresholding operator and gradient descent methods, which will be explained in [section gradient descent](#). Here is the soft thresholding operator for $X = I$

$$\begin{aligned}\hat{\theta}_i^{\text{Lasso}} &= S_{\lambda/2}(y) \\ &= \text{sgn}(y) \max(|y| - \lambda/2, 0)\end{aligned}\quad (12)$$

(Hjorth-Jensen, 2016, p. 91) The selection property makes the lasso the natural first tool when the number of parameters is large and most features are believed irrelevant. Solving the lasso regression problem requires a gradient descent method, which will be explained in [Gradient descent](#) and [Lasso](#)

E. Evaluation

1. Measures of quality

a. *Coefficient of determination* This

$$\begin{aligned}R^2 &= 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \\ \bar{y} &= \frac{1}{n} \sum_{i=1}^n y_i.\end{aligned}\quad (13)$$

b. *Mean squared error*

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2. \quad (14)$$

c. *Bias* For this project we will look at bias squared, the reason for this is that the squared values are positive, such that the terms will not cancel when we take the mean. the equation is.

$$\begin{aligned}\text{Bias}^2 &= \frac{1}{n} \sum_i (f_i - \mathbb{E}[\hat{y}])^2, \\ \mathbb{E}[\hat{y}] &= \frac{1}{n} \sum_i \hat{y}_i\end{aligned}\quad (15)$$

The bias measures how far the average prediction $\mathbb{E}[\hat{y}]$ is from the truth $f(x)$. (Hjorth-Jensen, 2016, p. 61-62)

In the case of this project, the function $f(x)$ is the runge function. The details on this will be discussed in the [bias variance tradeoff](#), since the data $y = f(x) + \epsilon$ includes noise.

d. *Variance* The variance is closely related to the standard deviation, which is the square root of the variance,

and tells how much the data varies within the sample.

$$\text{Var}[\hat{y}] = \frac{1}{n} \sum_i (\hat{y}_i - \mathbb{E}[\hat{y}])^2 \quad (16)$$

(Hjorth-Jensen, 2016, p. 62) The variance measures how much the prediction moves when the training set is redrawn. (Feil?)

2. Bias-Variance tradeoff

Before we perform an analysis of the bias variance tradeoff on the test data, we make a figure similar to Fig. 2.11 of Hastie, Tibshirani and Friedman. (Hastie et al., 2017, p. 38) This figure displays only the test and training MSE. The test MSE can be used to indicate possible regions of low/high bias and variance. To solve this, different values of noise, datapoints and polynomial degree is explored.

The bias variance tradeoff equation:

$$\begin{aligned}\mathbb{E}[(y - \hat{y})^2] &= \frac{1}{n} \sum_{i=1}^n (f_i - \mathbb{E}[\hat{y}_i])^2 \\ &\quad + \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - \mathbb{E}[\hat{y}_i])^2 + \sigma^2\end{aligned}\quad (17)$$

$$\text{MSE} = \text{Bias}^2 + \text{Var}[\hat{y}] + \text{irreducible noise} \quad (18)$$

Our empirical bias-related term compares the mean prediction with the noisy test targets, rather than with the noise-free function. Consequently, it also contains a contribution from the test noise and should not be interpreted as the squared bias alone.

a. *Fig 4.17*

3. Bootstrap

We study the bias variance tradeoff by implementing the bootstrap resampling technique. For studying this tradeoff we will only use ordinary least square for simplicity. The bootstrap approximates the sampling distribution of an estimator by repeatedly sampling from the observed data.

1. Draw with replacement n values from the observations $\mathbf{x} = (x_0, \dots, x_{n-1})$, forming a resample $\mathbf{x}^*$
2. Evaluate the estimator on the resample to obtain $\hat{\theta}^*$
3. Repeat k times

(Hjorth-Jensen, 2016, p. 65) The gist is that since $\hat{\theta} = \theta(\mathbf{X})$ is a function of random variables, theta itself is a random variable with some probability distribution function $p(t)$. The different bootstrap samples, pulls different random samples from the training data, with replacement, and each produce different fitted coefficients and therefore different predictions. The number of samples is chosen as a variable, in our case 40, 100 and 300 samples, and all the models are used to calculate the man squard error, bias and variance between all the fitted models and the test data. This is done for each degree of the polynomial. To pull the random data from the training data, we use sci-kit function `resample` for pulling the random bootstrap data.

a. *Bootstrap Pseudocode* LLM Declaration: This pseudocode is a copy of the reference project code c: Bootstrap (Holmen, 2026) created with help of OpenAI codex (OpenAI, nd)

Listing 4 Bootstrap estimation of prediction error, a bias-related term, and variance for OLS.

```
FUNCTION BootstrapBiasVariance(n_bs, maxdeg,
    seed):
    Initialize random generator using seed
    Generate n_bs bootstrap seeds

    error    <- zeros(maxdeg + 1)
    bias     <- zeros(maxdeg + 1)
    variance <- zeros(maxdeg + 1)

    FOR degree = 0, ..., maxdeg:
        X <- DesignMatrix(x, degree)

        Split X and y into:
            70% training data
            30% test data
        Use random seed 2026

        n_test <- number of test observations
        predictions <- zeros(n_test, n_bs)

        FOR b = 0, ..., n_bs - 1:
            Draw paired rows from X_train and
                y_train
                with replacement
                using bootstrap seed b
                and preserving the training set
                size

            X_bs, y_bs <- bootstrap sample

            theta <- pseudoinverse(X_bs) * y_bs

            predictions[:, b] <- X_test * theta

        mean_prediction <- row means of
            predictions
        point_variance <- row population
            variances
                of predictions

        targets <- repeat y_test in n_bs columns
```

```
error[degree] <- mean(
    (targets - predictions)^2
)

bias[degree] <- mean(
    (y_test - mean_prediction)^2
)

variance[degree] <- mean(point_variance)

RETURN error, bias, variance
```

4. Kfold Cross Validation

The data are divided into k roughly equal, mutually exclusive and exhaustive subsets, the folds. Each in turn plays the role of the test set while the union of the k-1 folds serve as the training set. Each observation is therefore used for testing exactly once and for training exactly k-1 times. The procedure is:

1. Shuffle the data set at random
2. Split into k folds.
3. For each fold: hold it out, fit the model on the remaining folds, evaluate on the held out folds and discard the model.
4. Summarize the model by the mean, the spread, of the k scores.

(Hjorth-Jensen, 2016, p. 66) The k fold cross validation is solved with closed form solutions for ridge and OLS, and with ski-kit functions Linearregression and Ridge. The mean value of the mean squared error of all folds is calculated analytically and with sci-kit function cross_val_score, and the difference between analytical and sci-kit solutions are calculated. 5 fold and 10 fold cross validation is used, and the ridge regression is calculated for different values of lambda, the penalty. The sci-kit method is used to ensure that the results agree with the results obtained numerically.

The cross validation is used to find the best degree, the best lambda value for ridge, and the best overall model, also the difference between the 5 fold and the 10 fold cross validation result is shown.

F. Gradient descent

Note: The chosen polynomial degree for gradient descent is 5 degrees.

1. Plain gradient descent

The gradient descent methods introduced is a method for finding the optimal parameter θ , and is thus a replacement for the closed form solutions of OLS and Ridge, and

gives a solution for the lasso regression problem, which had no analytical CF solution. In this regard the previous solutions for OLS and ridge can be used to verify that the gradient descent methods converge to the analytical solutions.

In its simplest form, the gradient descent method can be written

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} C(\theta_k) \quad (19)$$

(Hjorth-Jensen, 2026) The gradients of the cost function, will be solved both analytically and with automatic differentiation (JAX), and the results will be compared to prove that they agree to numerical precision.

Initially the learning rate η is guessed and adjusted for convergence over the number of iterations. But the learning rate has some limits, as long as the Hessian is known, as it is for Ridge and OLS, we can calculate this limit, the limit of convergence: For all directions of the cost function to converge we need $0 < \eta < \frac{2}{\lambda_{max}}$, where λ_{max} is the largest eigen value of the Hessian. Since the Hessian is the second derivative of the cost function, the largest eigenvalue is the steepest direction, and the smallest eigenvalue is the slowest direction, thus we can define the condition number, $\kappa = \frac{\lambda_{max}}{\lambda_{min}}$, which is proportional to the number of iterations needed to reduce the error by a fixed factor. The bound is set by the steepest direction, and a flat direction (0). Presumably the number of iterations reaches its minimum at $\eta = \frac{2}{\lambda_{min} + \lambda_{max}}$, and diverges exactly at $\frac{2}{\lambda_{max}}$.

Momentum, Adagrad, RMSprop and Adam are advanced gradient descent methods where the learning rate $\eta(n)$ is updated during the iteration. Opposed to plain gradient descent where the learning rate is constant. The more advanced methods are more suitable for cases where we cannot analytically calculate the Hessian of the cost function.

2. Pseudocode GD

LLM Declaration. This pseudocode is a pseudo copy of the reference project code *f: Momentum, Adagrad, RMSprop and Adam* (Holmen, 2026) Created with help of OpenAI codex (OpenAI, nd) with latex package `lstlisting`.

The following two pseudocodes show simplified interfaces for the optimisation loop and a single update. The update step has access to the learning rate, iteration number, method settings and stored optimiser state. The stopping criterion includes a maximum number of iterations.

Listing 5 Optimisation loop: optimise.

```

INPUT method,  $\theta_0$ , gradient,  $\gamma$ 

 $\theta \leftarrow \theta_0$ 
history  $\leftarrow$  [copy of  $\theta$ ]
Set learning rate to  $\gamma$ 
Initialise method state and iteration count

REPEAT until stopping criterion is met:
    grad  $\leftarrow$  gradient( $\theta$ )
     $\theta \leftarrow$  optimiser_step(method,  $\theta$ , grad)
    Append a copy of  $\theta$  to history
    Advance iteration count

RETURN history

```

Listing 6 Single parameter update: optimiser_step.

```

INPUT method,  $\theta_n$ , grad

Apply the selected method's update rule
to obtain  $\theta_{n+1}$  and update method state

RETURN  $\theta_{n+1}$ 

```

Listing 7 Momentum.

```

INPUT C,  $\theta_0$ , T,  $\gamma > 0$ ,  $0 \leq \beta < 1$ 

 $v_0 \leftarrow \mathbf{0}$ 

FOR  $t = 1, \dots, T$ :
     $g_t \leftarrow \nabla C(\theta_{t-1})$ 
     $v_t \leftarrow \beta v_{t-1} + \gamma g_t$ 
     $\theta_t \leftarrow \theta_{t-1} - v_t$ 

RETURN  $\theta_T$ 

```

Listing 8 AdaGrad.

```

INPUT C,  $\theta_0$ , T,  $\gamma > 0$ ,  $\varepsilon > 0$ 

 $r_0 \leftarrow \mathbf{0}$ 

FOR  $t = 1, \dots, T$ :
     $g_t \leftarrow \nabla C(\theta_{t-1})$ 
     $r_t \leftarrow r_{t-1} + g_t^2$ 
     $\theta_t \leftarrow \theta_{t-1} - \frac{\gamma}{\sqrt{\varepsilon + r_t}} g_t$ 

RETURN  $\theta_T$ 

```

Listing 9 RMSProp.

```

INPUT C,  $\theta_0$ , T,  $\gamma > 0$ ,  $0 \leq \rho < 1$ ,  $\varepsilon > 0$ 

 $r_0 \leftarrow \mathbf{0}$ 

FOR  $t = 1, \dots, T$ :
     $g_t \leftarrow \nabla C(\theta_{t-1})$ 
     $r_t \leftarrow \rho r_{t-1} + (1 - \rho) g_t^2$ 
     $\theta_t \leftarrow \theta_{t-1} - \frac{\gamma}{\sqrt{\varepsilon + r_t}} g_t$ 

RETURN  $\theta_T$ 

```

Listing 10 Adam.

```

INPUT  $C, \theta_0, T, \gamma > 0,$ 
          $0 \leq \beta_1, \beta_2 < 1, \varepsilon > 0$ 

 $m_0 \leftarrow \mathbf{0}$ 
 $r_0 \leftarrow \mathbf{0}$ 

FOR  $t = 1, \dots, T$ :
     $g_t \leftarrow \nabla C(\theta_{t-1})$ 
     $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
     $r_t \leftarrow \beta_2 r_{t-1} + (1 - \beta_2) g_t^2$ 
     $\hat{m}_t \leftarrow \frac{m_t}{1 - \beta_1^t}$ 
     $\hat{r}_t \leftarrow \frac{r_t}{1 - \beta_2^t}$ 
     $\theta_t \leftarrow \theta_{t-1} - \frac{\gamma}{\sqrt{\hat{r}_t} + \varepsilon} \hat{m}_t$ 

RETURN  $\theta_T$ 

```

3. Lasso Gradient descent

The Lasso cost function is the mean squared error plus an L1 penalty. The intercept θ_0 is not penalised:

$$C_{\text{Lasso}}(\theta) = \frac{1}{n} \|X\theta - y\|_2^2 + \lambda \sum_{j=1}^{p-1} |\theta_j|. \quad (20)$$

Here, p is the number of coefficients, including the intercept.

Since the L1 penalty is not differentiable at zero, we use a subgradient:

$$g_{\text{Lasso}}(\theta) = \frac{2}{n} X^T (X\theta - y) + \lambda s(\theta). \quad (21)$$

The first term is the MSE gradient. The penalty contribution is defined by $s_0 = 0$ and, for $j = 1, \dots, p-1$,

$$s_j = \begin{cases} -1, & \theta_j < 0, \\ 0, & \theta_j = 0, \\ 1, & \theta_j > 0. \end{cases} \quad (22)$$

At zero, we choose $s_j = 0$, which is a valid subgradient of the absolute value from the interval $[-1, 1]$. This will be the input for the optimization pseudocode explained in gradient descent, and thus the optimal parameters will be found through gradient descent.

G. Experimental setup

LLM Declaration: This section was generated with the assistance of OpenAI Codex (OpenAI, nd), which analyzed the `FullProject.ipynb` (Holmen, 2026) code structure and helped organize the description of the experimental setup.

We generate 350 observations from the Runge function, $f(x) = 1/(1 + 25x^2)$, with x sampled uniformly from $[-1, 1]$ and Gaussian noise with standard deviation $\sigma = 0.25$. We use random seed 2026 and a 70/30 training–test split. Polynomial features are standardised using training-set statistics, except in the bias–variance and bootstrap experiments. The intercept is excluded from regularisation.

OLS and Ridge are evaluated for polynomial degrees 0–15, with Ridge penalties ranging from 10^{-3} to 10^2 . The bias–variance analysis extends to degree 25. Bootstrap experiments use 40, 100 and 300 resamples. Shuffled 5-fold and 10-fold cross-validation compare OLS, Ridge and Lasso for degrees 0–14 and several regularisation strengths. Performance is measured using MSE and R^2 , with mean validation MSE used for model comparison.

The optimisation experiments use degree 5 and a regularisation strength of 0.01 for Ridge and Lasso. Analytical gradients are compared with automatic differentiation using JAX. Plain gradient descent, Momentum, AdaGrad, RMSProp and Adam are evaluated at several learning rates. Lasso uses a subgradient of the L_1 penalty. Convergence is assessed against analytical OLS/Ridge solutions and a scikit-learn Lasso reference, using parameter error, iteration count and execution time.

Finally, OLS, Ridge and Lasso are compared using shuffled 5-fold and 10-fold cross-validation for polynomial degrees 0–14. Several regularisation strengths are tested for Ridge and Lasso. Features are standardised within each training fold. The models are compared using mean validation MSE and its standard deviation across folds, selecting the degree and regularisation strength with the lowest mean MSE.

III. RESULTS

A. The data

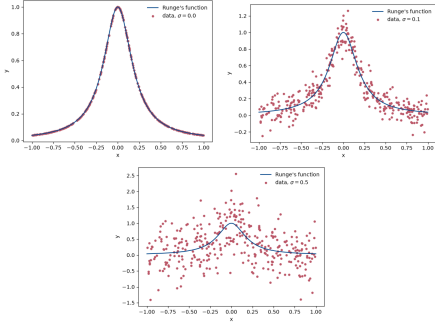


FIG. 1 Data generated from the Runge function with different noise levels

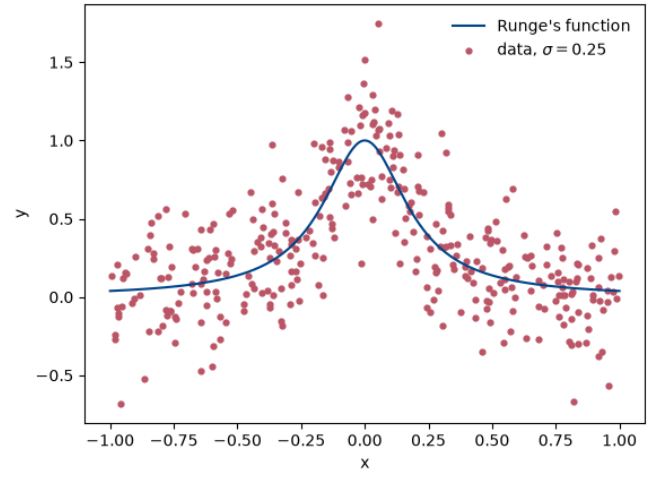


FIG. 2 Final data used during project

B. OLS and Ridge regression

1. OLS

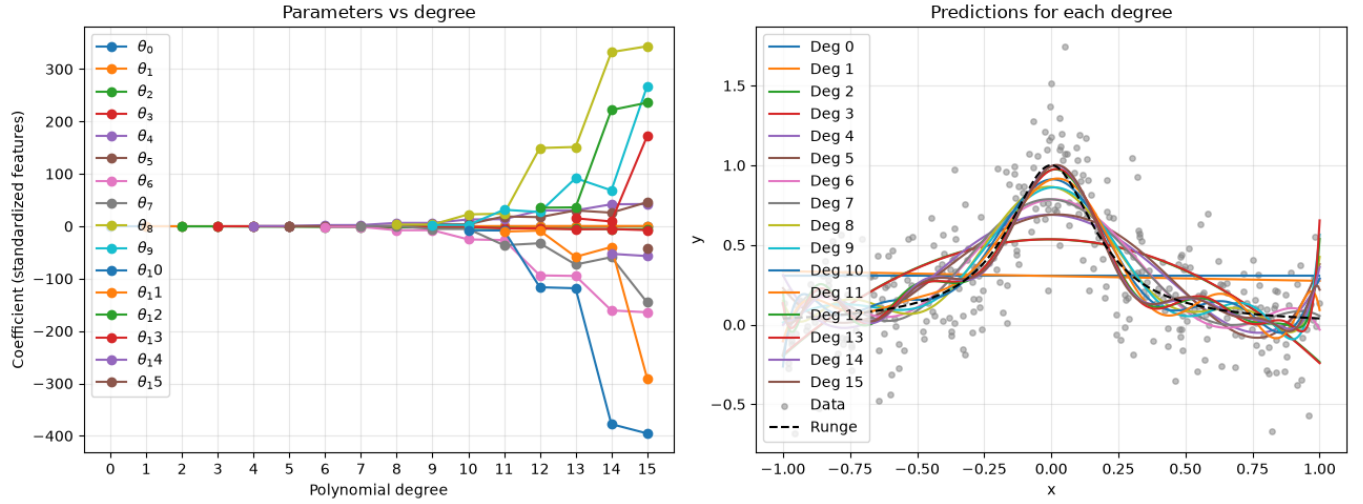


FIG. 3 Model predictions and parameters vs Degree

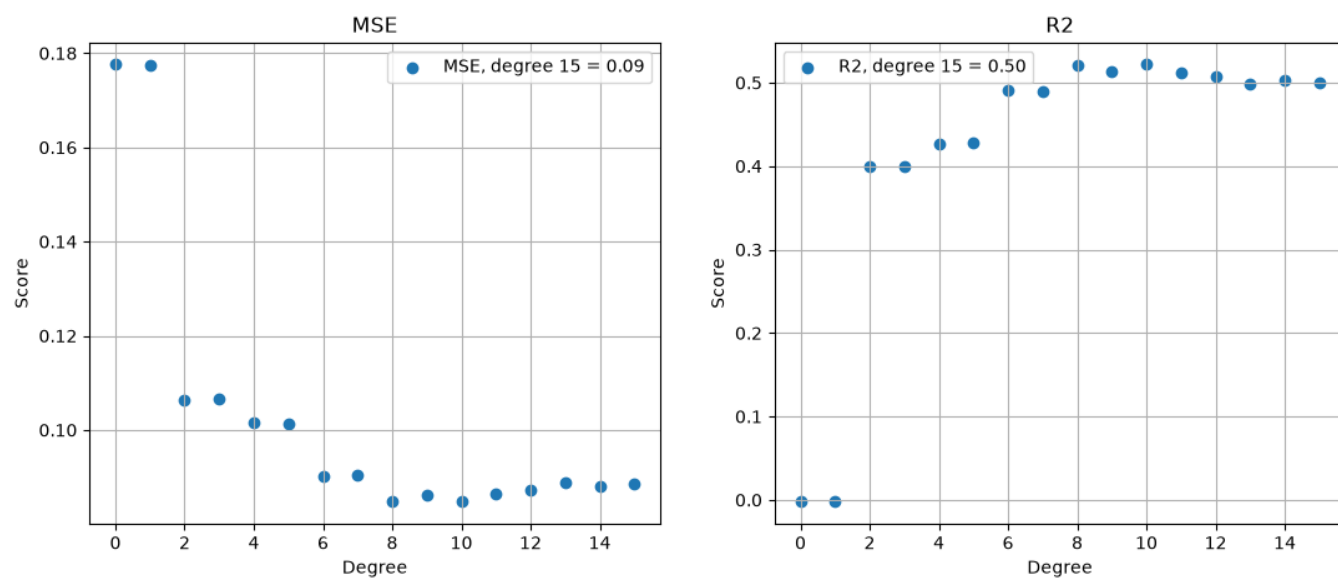


FIG. 4 Measures of quality

2. Ridge

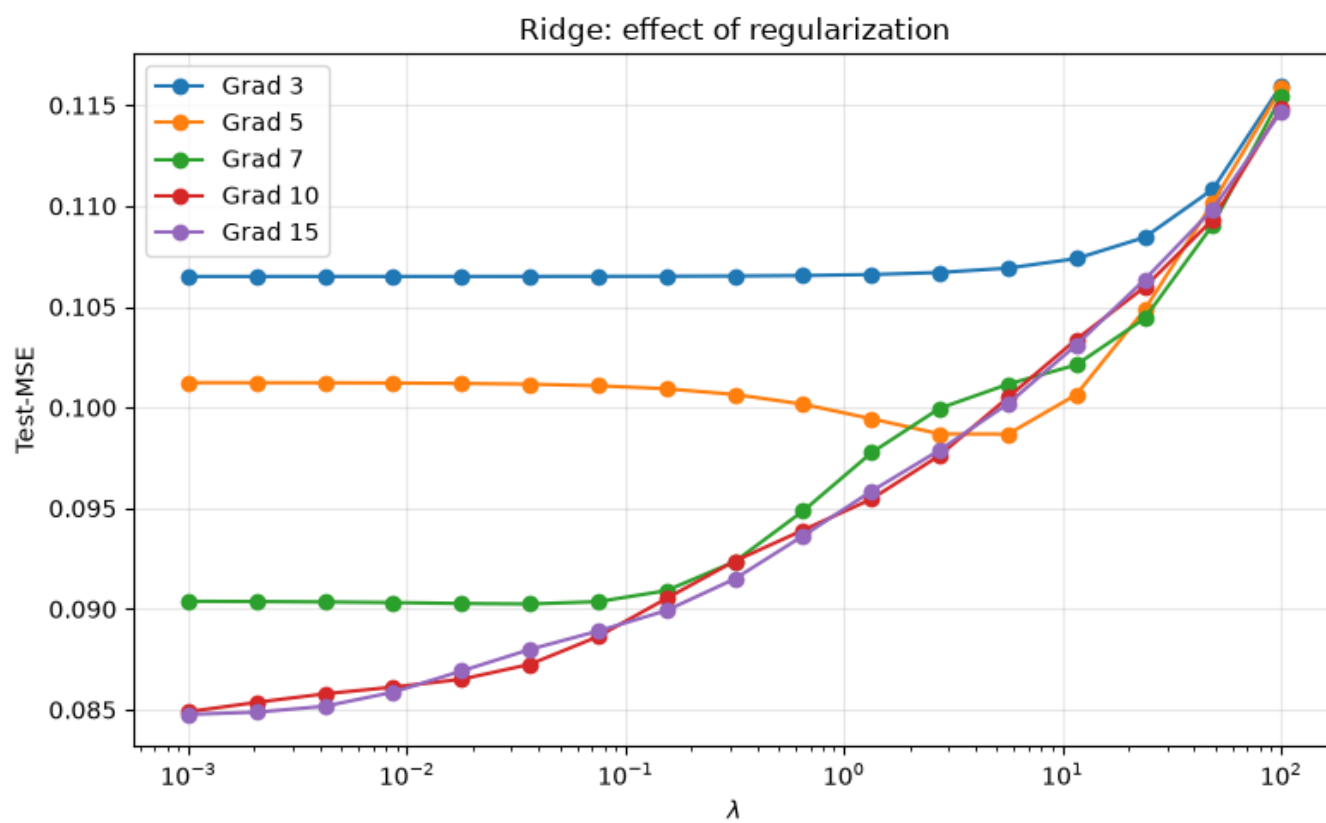


FIG. 5 Regularization vs lambda

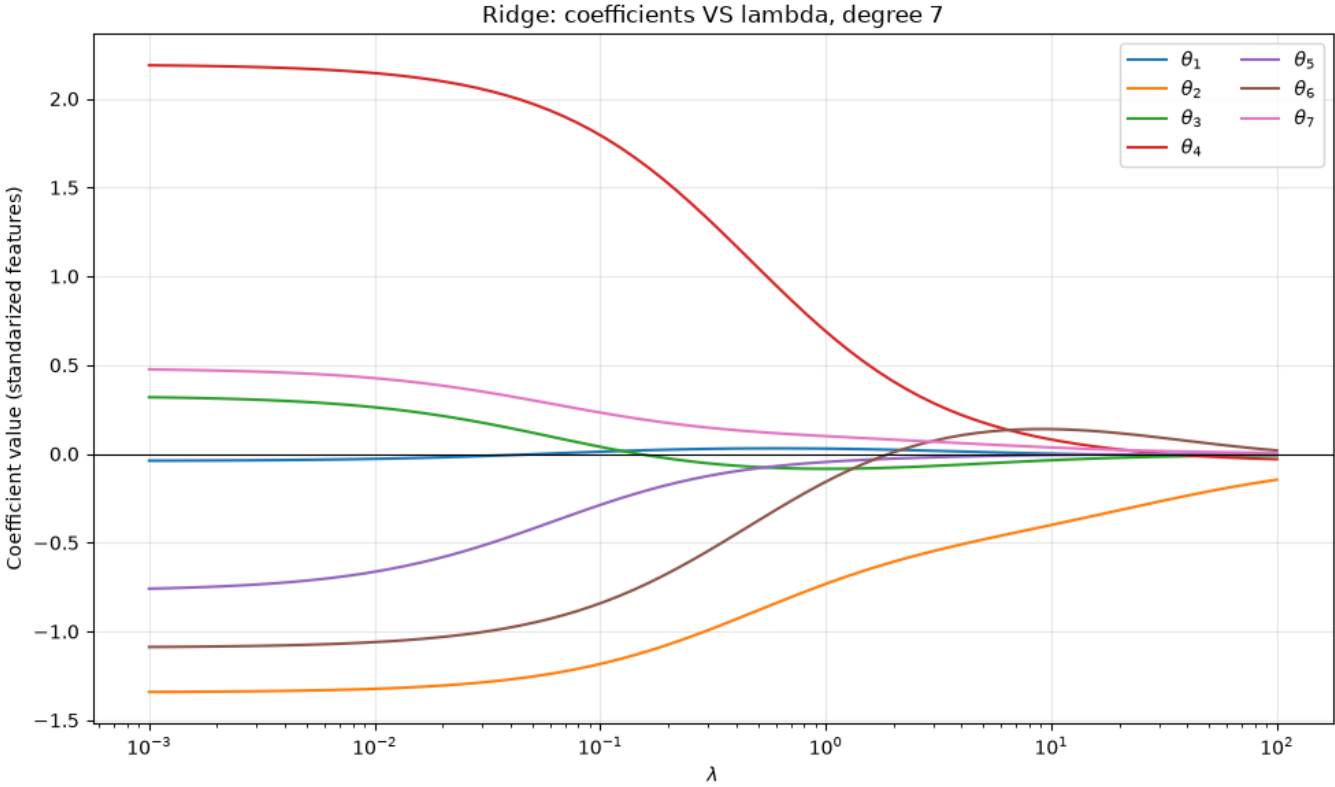


FIG. 6 Coefficient values vs lambda

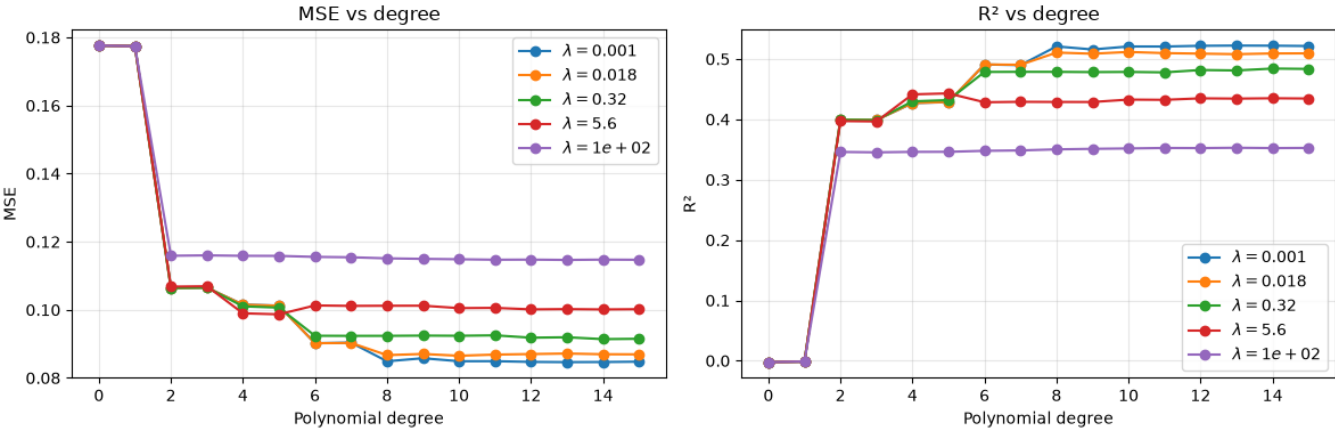


FIG. 7 Measures of quality

C. Bias-Variance tradeoff and bootstrap

1. Fig 2.11

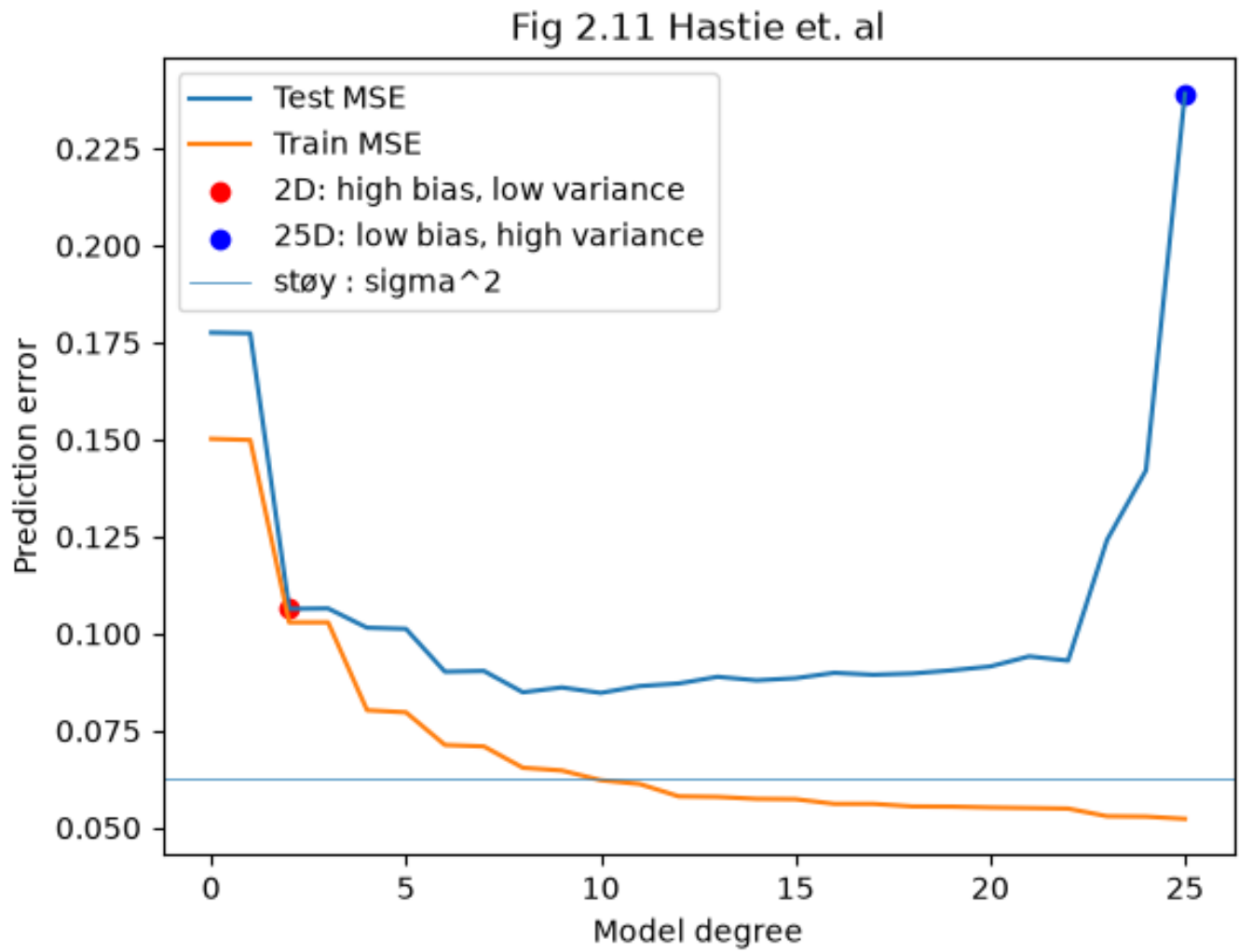


FIG. 8 Bias-Variance Reference figure 2.11

2. Bootstrap

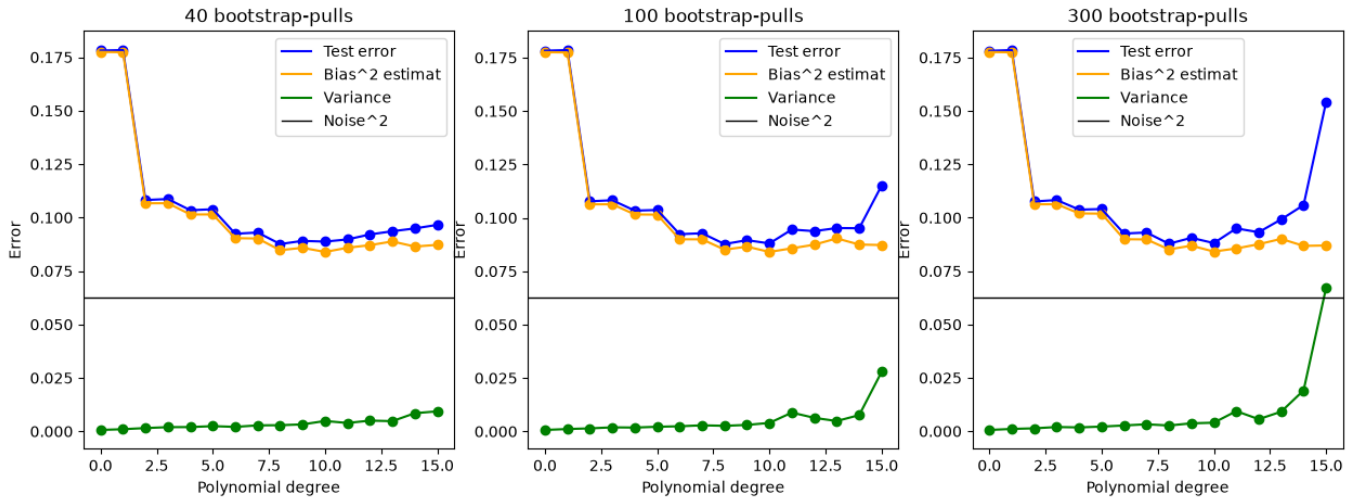


FIG. 9 Bias-variance tradeoff: Bootstrap

D. K-fold Cross validation

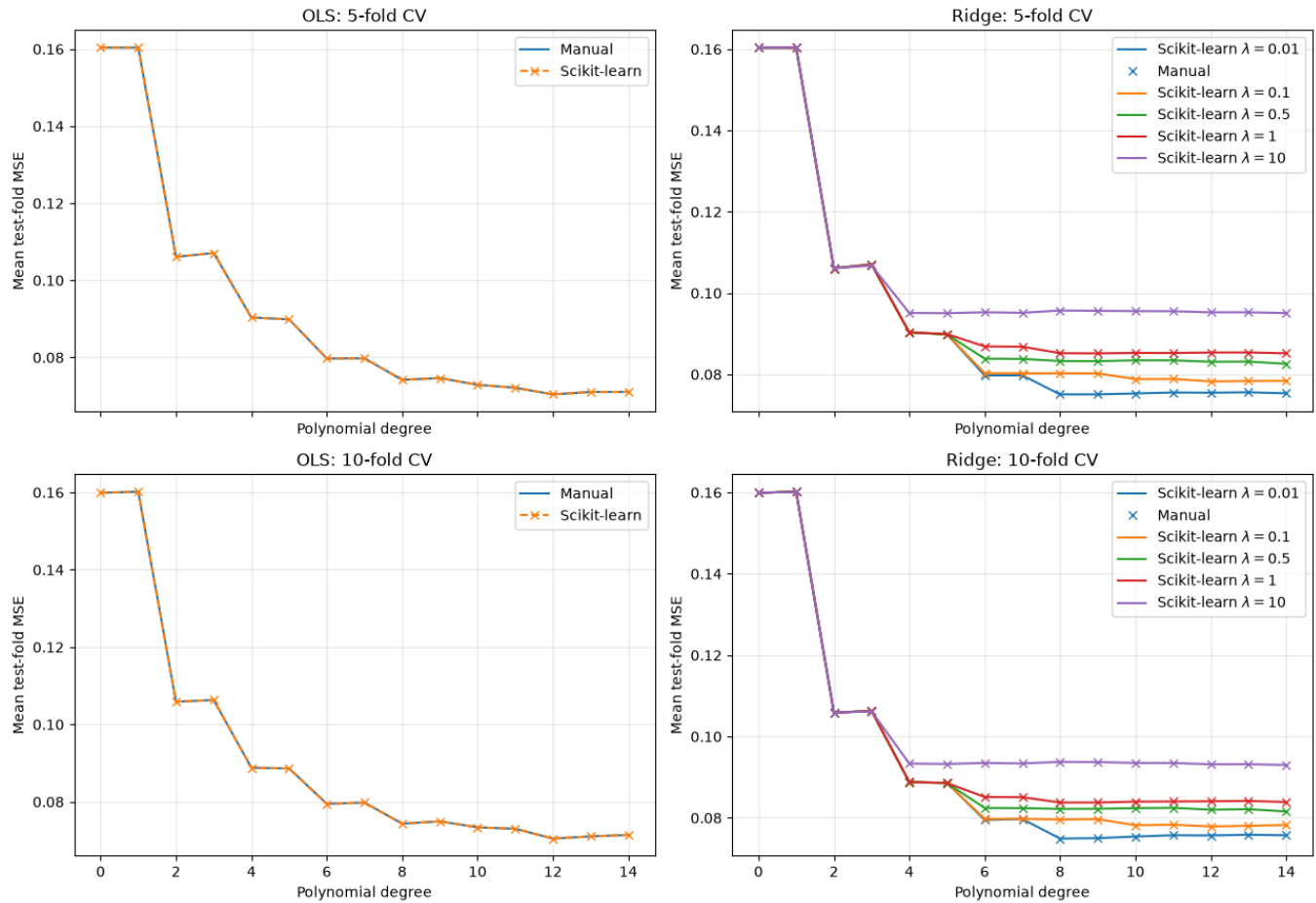


FIG. 10 Kfold Cross validation

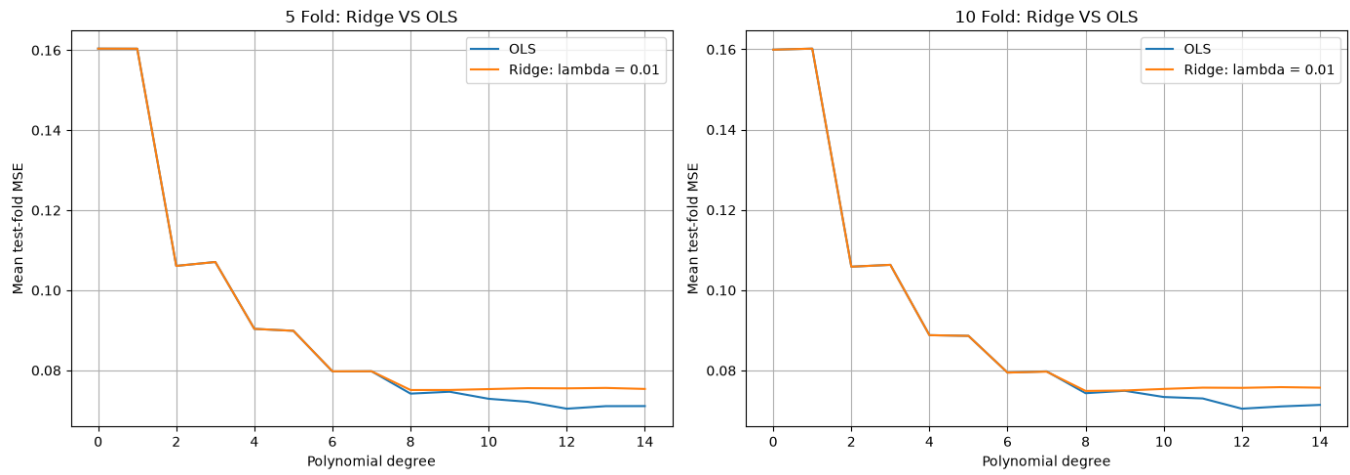


FIG. 11 Kfold: Ridge vs OLS

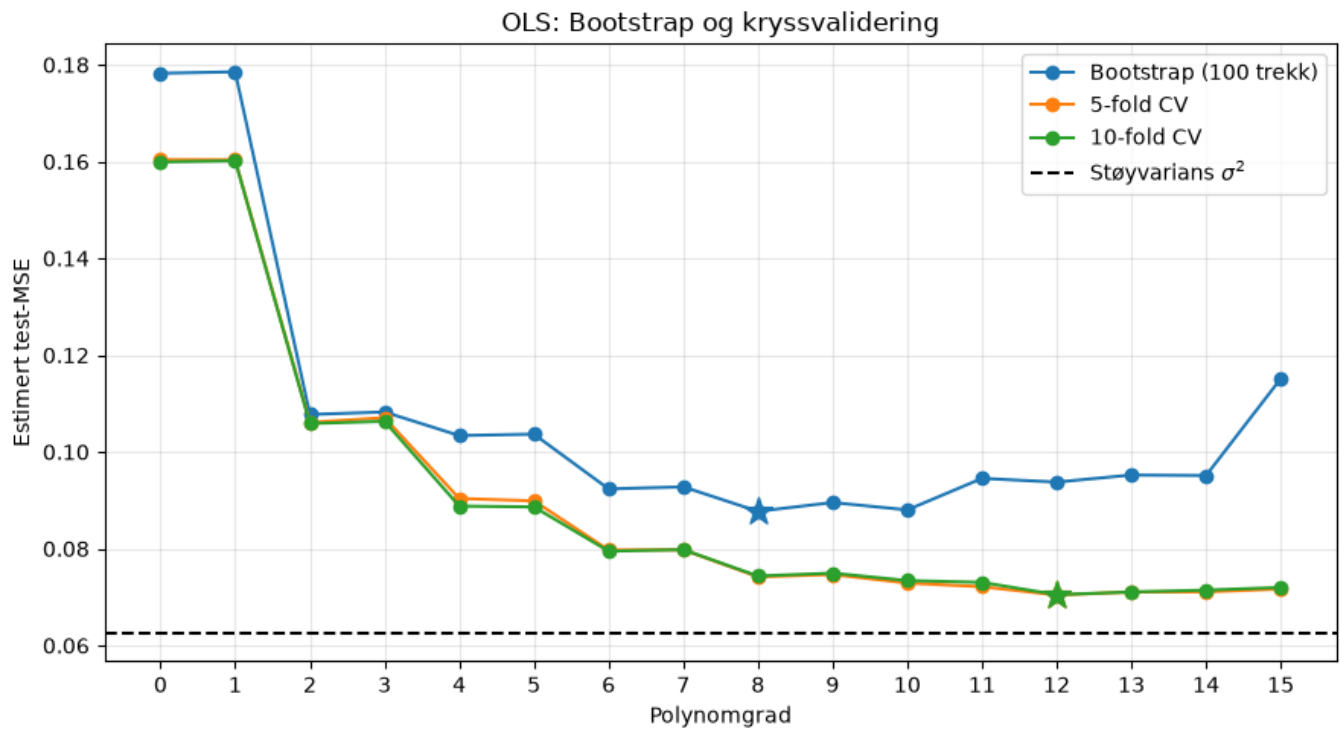


FIG. 12 Kfold and Bootstrap

TABLE I Final result K-fold and Bootstrap cross validation

Modell	Best degree	MSE
Bootstrap(100)	8	0.088
5-fold CV	12	0.070
10-fold CV	12	0.070

E. Gradient descent and optimisation methods

1. Automatic differentiation

The analytical gradients for OLS and Ridge were compared with automatic differentiation using jax, using polynomial degree 5 and $\lambda = 0.01$. On the full, unscaled dataset, the maximum absolute difference between gradi-

ent components was 1.332×10^{-15} . This difference shows numerical percision and indicate agreement between the gradient both gradient solutions.

The comparison was repeated on the scaled training data at two different start parameter vectors. The differences ranged from 8.882×10^{-16} to 1.332×10^{-15} , showing numerical precision also on the scaled training data, which will be used throughtout the gradient descent.

2. Plain Gradient descent

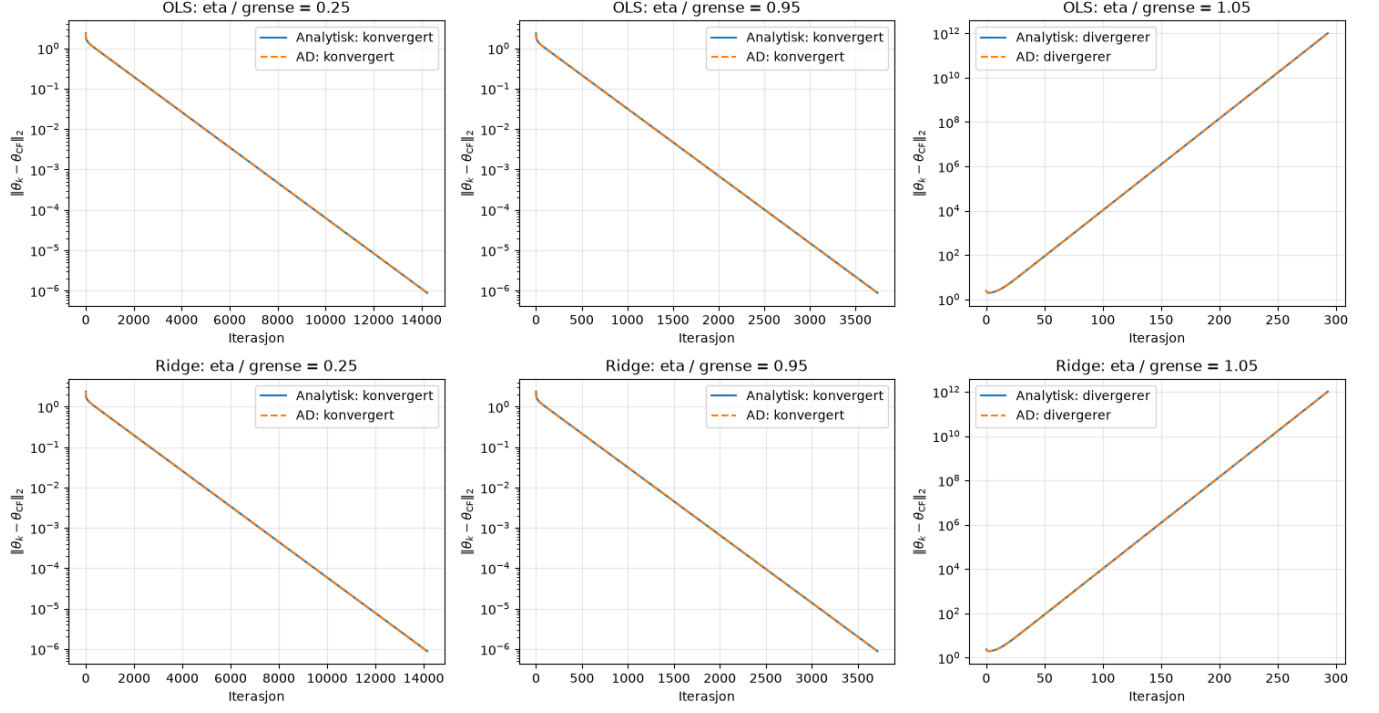


FIG. 13 Plain gradient descent for OLS and Ridge with different learning rates.

3. Advanced gradient descent

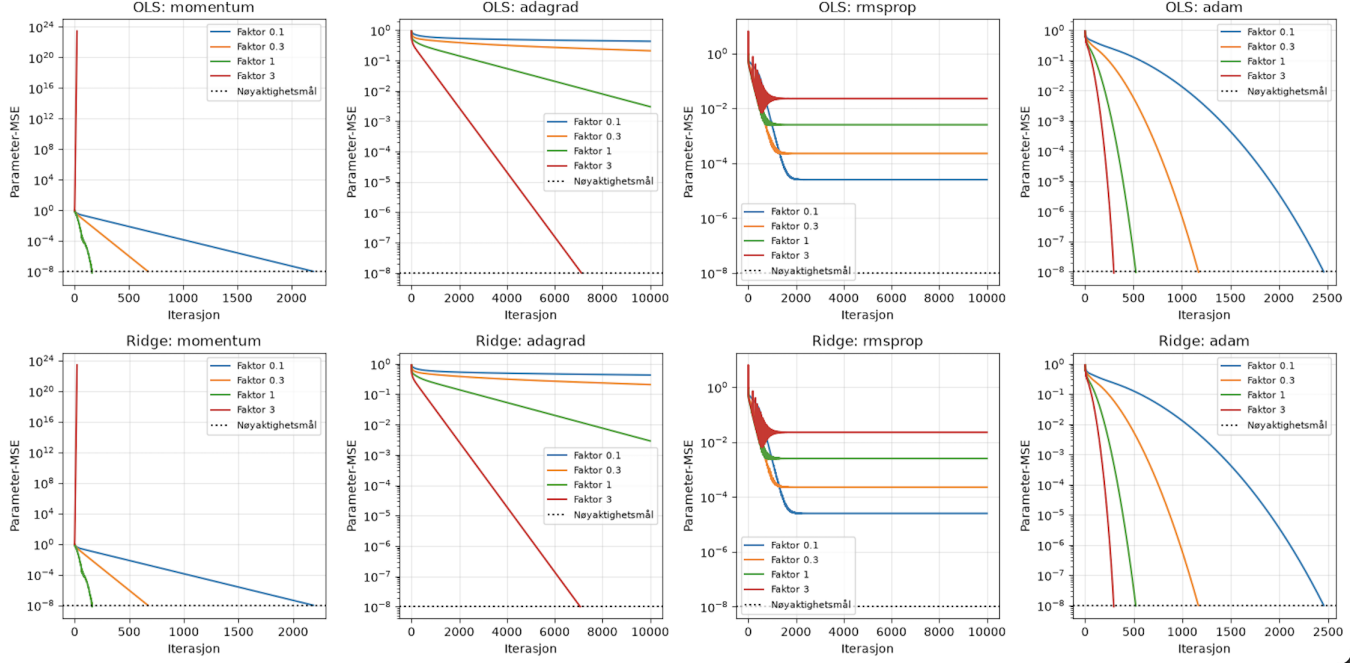


FIG. 14 Advanced gradient descent methods with OLS and ridge

The learning-rate limits for plain gradient descent are:

- OLS: $\eta_{\lim} \approx 0.356726$.
- Ridge: $\eta_{\lim} \approx 0.356721$.

TABLE II Base learning rates for the optimisation methods.

Method	OLS	Ridge
Momentum	0.338890	0.338885
AdaGrad	0.100000	0.100000
RMSPprop	0.100000	0.100000
Adam	0.356010	0.356000

For Momentum, the base rate is $0.95 \times \eta_{\lim}$. Each base rate is multiplied by factors $\{0.1, 0.3, 1, 3\}$ to investigate learning-rate sensitivity.

F. Lasso with gradient descent

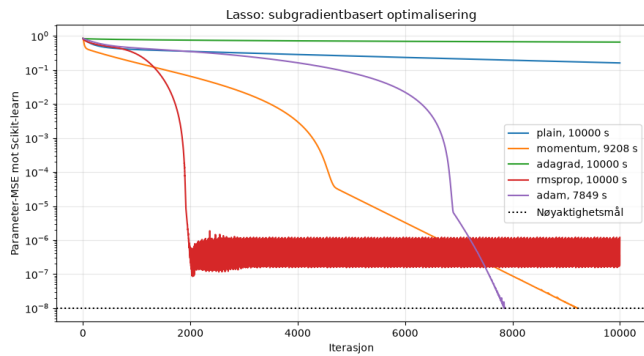


FIG. 15 Lasso GD with $\eta=0.01*0.1$

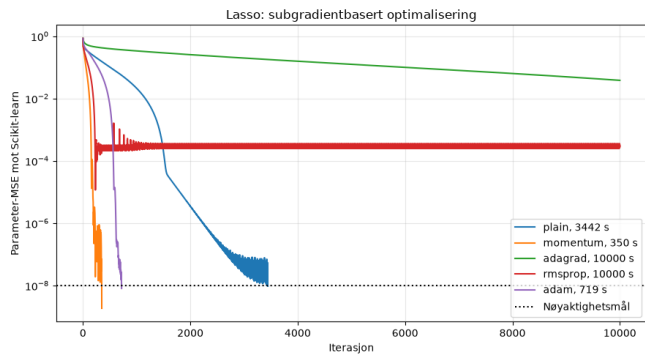


FIG. 18 Lasso GD with $\eta=0.01*3$

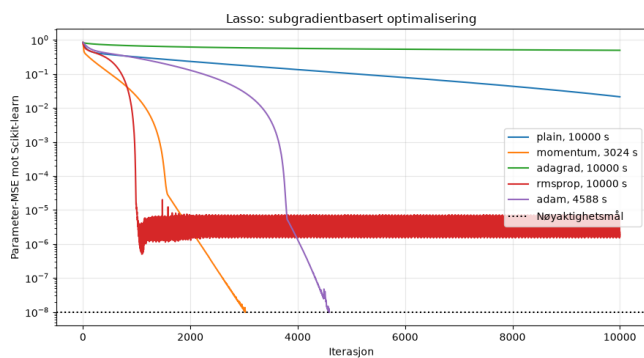


FIG. 16 Lasso GD with $\eta=0.01*0.3$

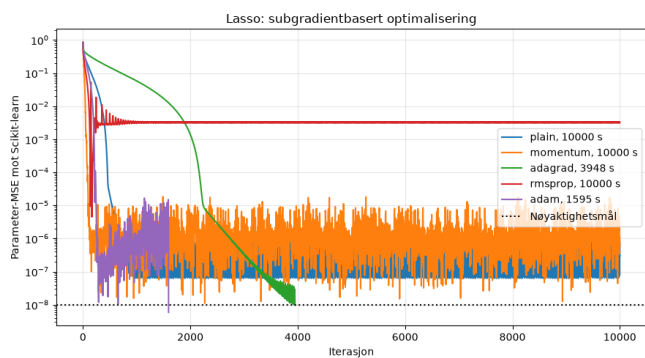


FIG. 19 Lasso GD with $\eta=0.01*10$

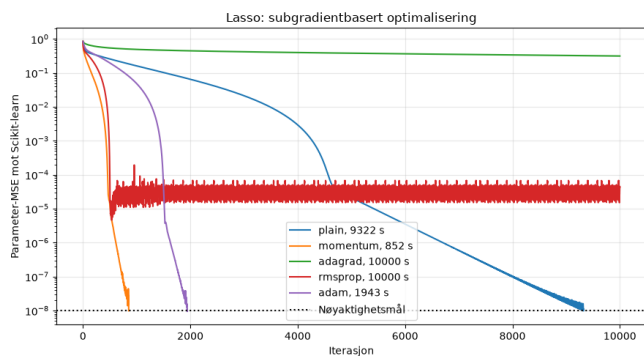
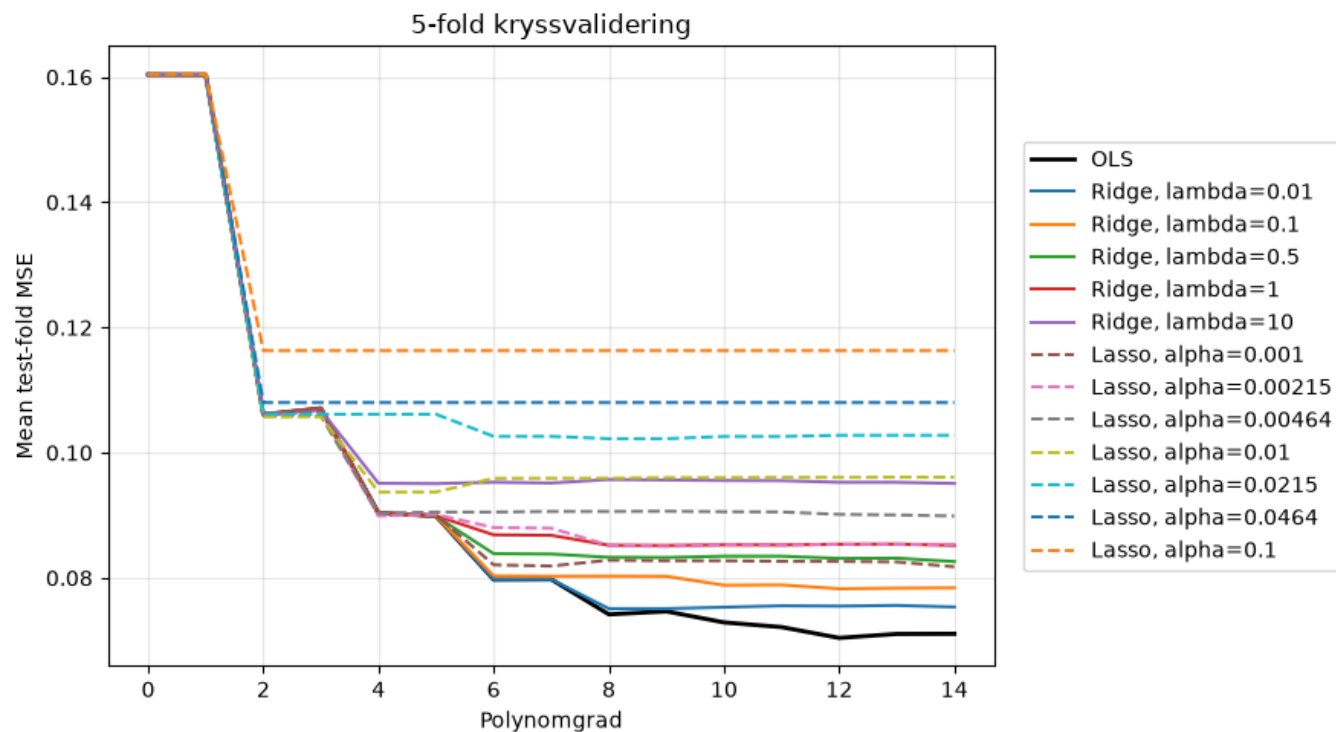
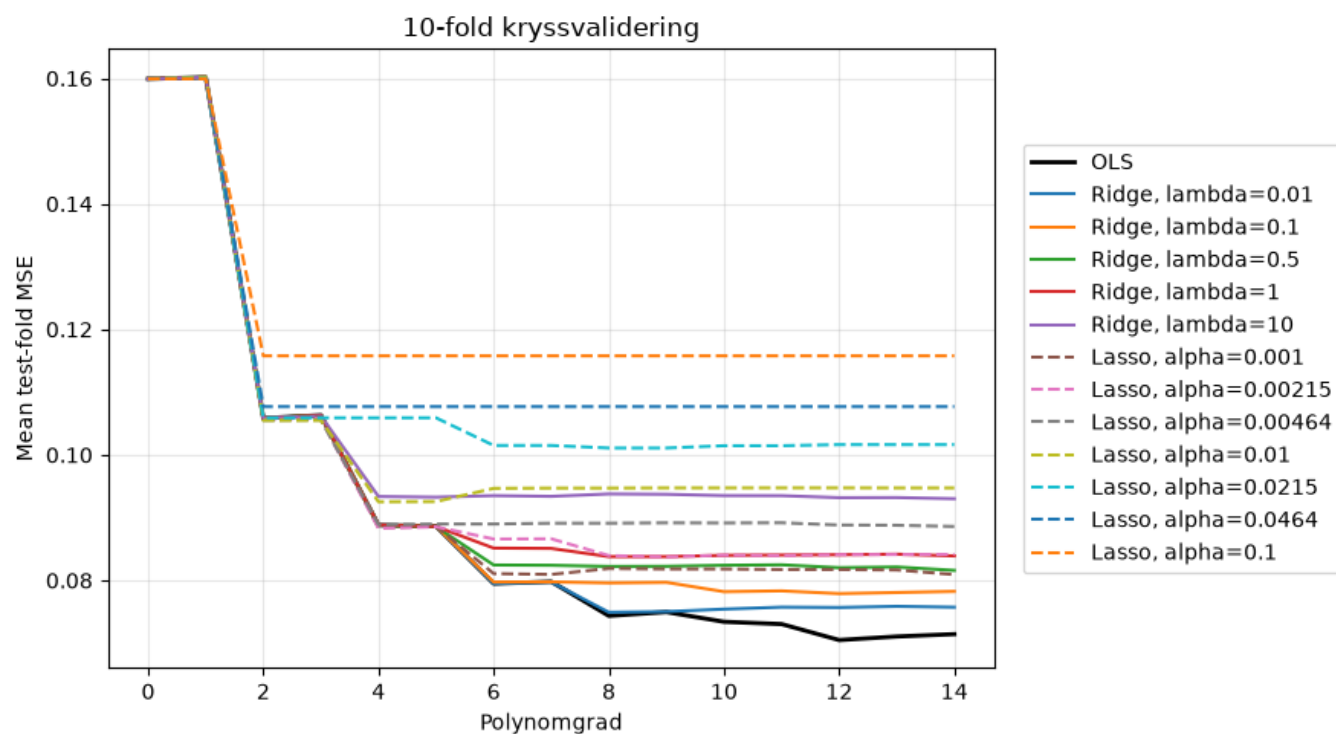


FIG. 17 Lasso GD with $\eta=0.01*1$

G. Final cross-validation comparison

FIG. 20 Final $k=5$ fold CV for OLS, Ridge and LassoFIG. 21 Final $k=10$ fold CV for OLS, Ridge and Lasso

IV. DISCUSSION

A. OLS

1. Parameter plot

Figure 3 shows the OLS parameter plot. A couple things to notice is that the parameter figure does not show the lower degree polynomial coefficients, because the higher degree coefficients blow up in size as the degree increases; this size difference hides the lower degree coefficients in the plot. The higher degree coefficients are explained by the figure on the left, which shows large oscillations around the endpoints $x = -1, 1$. This is not the case for the lower degree polynomial fits, as these do not have enough complexity to create such oscillations. The simplest degree shown in the right-hand figure is the degree 3 polynomial fit, which is a parabola with no oscillation around the endpoints. Strangely enough, the degree 2 polynomial is not visible in this right-hand figure; it should be a green parabola. Exploring this further:

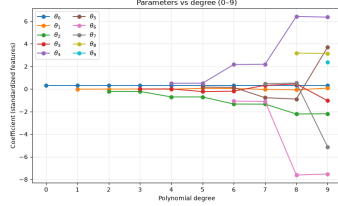


FIG. 22 Parameter plot cut at 9 degrees for scaling

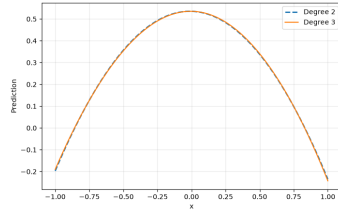


FIG. 23 Degree 2 vs 3 Polynomial fit

As expected, the degree 2 and 3 polynomial fits overlap. The degree 0 coefficient is close to zero, and probably only affected by the noise. The degree 1 coefficient shows a negative trend, this may be a result of the random noise being slightly negatively inclined.

2. Measure of quality

Figure 4 show the measure of quality plot. The MSE decrease rapidly from a first to second degree polynomial, and increases slightly for third degree, this fits with the previous discussion, where the second and third order fit overlapped. The MSE and R2 plots show correspondence, a low MSE corresponds to a high R2 value, the best MSE and R2 score is for an 8 degree polynomial fit, which has the lowest MSE and highest R2 score. Higher

degrees don't really improve the models quality in terms of MSE and R^2 . The best possible R2 score is 1, which means the prediction hits all the true test values, the R2 score in our fit has a limit around 0.5, which means that overall half the predicted points hit the target value, this may be a result of the noise ratio $\sigma = 0.25$ and the form of the runge function, as seen in the coefficient plot in figure 3 and discussed above.

B. Ridge

1. Regularization

Figure 5 and Figure 6 show the effect of the regularization on test-MSE as a result of different regularization parameters λ . Figure 5 shows the effect on degrees 3, 5, 7, 10 and 15, and show that the minimum MSE is caused by different λ values for each degree. Degree 5 has a minimum around $\lambda = 0.5$, degree 10 and 15 for $\lambda = 0.001$. The trend seems to be that higher degree polynomial fits need weaker penalty values to reach a better fit (Lower test-MSE). By zooming in on degree 7, as in Figure 6, we see how the different coefficients are regularized by an increasing λ value. We also see that a high penalty $\lambda = 100$ shrinks all the coefficients close to zero, which proves the initial assumption for ridge regression. Testing showed that below 0.001 no positive effect on Test-MSE happened, for all degrees.

2. Measure of Quality

Figure 7 show the measure of quality plots for ridge, both MSE and coefficient of determination. The plots show the same correspondence as OLS, and the R^2 limit is still around 0.5. What we see of importance here is that these results further prove the result in figure 5 and 6, the relation between weak penalty and high degree polynomial fit. We see the plateau where an increasing polynomial degree don't change the MSE or R^2 for the same λ value. As if a high λ affects the higher degree coefficients more strongly than the lower degree coefficients, this is further shown in Figure 6 and Figure 3 Left plot, where the lower degree coefficients θ_i , are larger and less affected by λ , as θ_2 in Fig. 6, opposed to θ_7 ,

C. Bias-Variance tradeoff and Bootstrap

Figure 8 shows the bias variance tradeoff in effect, the max degree was tuned to 25 degrees to really show this effect of increasing test error by increasing the degree. This increasing test error is the result of high variance, overfitting. When both training and test error is high, this is a result of underfitting, low bias, in this case the variance is lower, which is indirectly seen from the low difference in MSE between test and training error. The bootstrap figure 9 shows the decomposition of the MSE into $bias^2$ and $variance$. Here we see the bias decreasing for higher degrees, which is good. The variance increase for higher degrees, which is expected. The difference between the test error and bias, and the variance is from the noise which is baked into the data, and the model cannot predict, lowering the noise would lower the test error as the bias decrease.

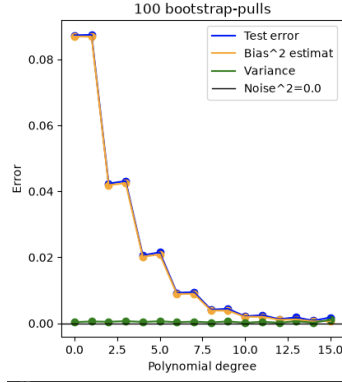


FIG. 24 Bias-Variance tradeoff, Bootstrap, 0 noise

D. K-fold Cross validation

Figure 10 show two things, first that the ski-kit learn kfold function agrees with the manual kfold function to numerical precision. Secondly, that there is not a noticable difference between 5fold and 10fold cross validation. The OLS CV plot show that the best polynomial degree for OLS is 12 degrees, 8 degrees is second best and can be chosen for simplicity and lower variance. The Ridge plot shows that the best degree and penalty is 8 degrees with $\lambda = 0.01$ Comparing both the best Ridge model (0.01) and OLS in Figure 11 shows that over 8 degrees OLS gives a better prediction in terms of test MSE than Ridge, for 8 degrees they are almost equal. Interestingly, Figure 12 and Table I for OLS comparing bootstrap and kfold, the bootstrap shows that 8 degrees gives the lowest MSE, while the kfold CV shows that 12 degrees gives the lowest MSE. This may indicate that kfold crossvalidation lowers the variance of the model.

E. Gradient Descent

1. Plain Gradient descent

Figure 13 show that the gradient descent converges below the limit $\frac{2}{\lambda_{max}}$ and diverges above it, can also see that close to the limit, $0.95 \cdot \lim$, the solution reaches the convergence limit of 10^{-6} for 3500 iterations, while for $0.25 \cdot \lim$, it takes 14000 iterations.

2. Advanced gradient descent

Figure 14 shows how the different advanced methods act for different learning rates. The limit for converge set is 10^{-8} . Adam is most robust, converging for all learning rates set, momentum converges for all learning rates except $3 \cdot 0.95 \cdot \eta_{lim}$. The best learning rate for adam is 3×0.356 , this learning rate gives the least iterations for convergence, around 300. The best learning rate for momentum is $1 \cdot 0.95 \cdot \eta_{lim}$, this learning rate converges around 200 iterations, and thus the best optimization method is momentum. Adagrad and rmsprop did not converge sufficiently for the set training rates, except adagrad for a learning rate of $3 \cdot 0.1$, with 7000 iterations. For a convergence limit of 10^{-4} rmsprop would have converge with a learning rate of $0.1 \cdot 0.1$. By increasing the total number of iterations, adagrad would probably also converge for the lower learning rates, as seen from the negative inclination of the graphs in the figure. One obvious issue with rmsprop and adagrad, is that both methods received lower base learning rates than adam and momentum, as shown in Table II.

The RMSprop may be a result of either a low initial learning rate, or a fault in the optimization algorithm itself.

F. Lasso gradient descent

Figure 15 to Figure 19 show the activity of the different methods in more detail, while solving for the optimal parameters. The best overall learning rate is $\eta = 0.03$, which is Figure 16, and the best method is momentum, which converges to the limit 10^{-8} after 350 iterations. The last figure, Figure 19 show what happens with the different methods when the learning rate is set too high. They oscillate around the minimum point, since the step size is too large, never reaching the required converge limit.

G. Final analysis

The final figures Figure 20 and Figure 21, show that the best method for solving the optimization problem is OLS, followed closely by ridge, and then Lasso. The reason for this may be the simple form of the runge function, which

allows for OLS to solve this problem with best accuracy, in terms of MSE.

V. CONCLUSION

The best method for solving this optimization problem is OLS, which is seen by the black line describing the OLS regression MSE for different degrees in Figure 21. The best degree is 12, but 8 degrees can also be chosen for model simplicity. For 8 degrees, ridge with $\lambda = 0.01$ have approximately the same test MSE as OLS, and can be chosen as well. The best optimization method for gradient descent, in terms of accuracy and cost, is momentum with learning rate $\eta = 0.338$, which is $\frac{2}{\lambda_{\max}(H)}$ for both OLS and Ridge.

VI. LLM DECLARATION

Did not create a sufficient LM declaration, to document the full use of Codex, with OpenAIs *GPT_6.1Sol*. The main algorithms are created with information from

(Hjorth-Jensen, 2016), and weekly exercises. AI was mainly used to display and store data, and for ensuring that the code ran efficiently. For the lasso regression code, codex created the skeleton with inspiration from the lecture notes. Throughout the full project code there is an LLM declaration in the code cells where codex is used. Codex was used for writing some of the text in the report, such as the [Section of the report](#). For the next report, the lesson is to do the right LLM declaration during the process of writing the code and report.

REFERENCES

- T. Hastie, R. Tibshirani, and J. Friedman (2017), *The Elements of Statistical Learning* (Springer).
- M. Hjorth-Jensen (2016), *Machine Learning and the Physical Sciences* (Springer).
- M. Hjorth-Jensen (2026), “Project 1 on Machine Learning,”.
- T. Holmen (2026), “Prosjekt1,” <https://github.com/Timholmen96/FYS-STK4155>.
- OpenAI (n.d.), “Codex,”.